

Orchard Core for Enterprise Systems

Practical Enterprise patterns for building modules, data, apis and integrations.

Niraj Soni

Technical lead and architect

Founder & President,
Surevelox

Orchard Harvest 2026

10–11 September 2026

About me

Niraj Soni

Technical lead and architect

Founder & President Surevelox, Inc



Helped enterprises building line-of-business systems on Orchard Core framework.

Helped build modular architecture, multi-tenancy, external identity and integrations and portals

Helped building solutions for enterprises in Manufacturing, Product Engineering, Insurance, Banking, Supply chain management, Independent software vendors and communication domains and Agencies.

This session is NOT

01	No Hands On Lab	No Coding exercise
02	No Code to share	There is no code sample
03	Not a sales pitch for OC / Surevelox	Not selling Orchard Core 😊
04	Not sharing something new about OC	Not sharing anything new that you don't know about Orchard Core
05	It's about Enterprise perspective	Its about value benefit for Enterprise, The hidden GEM

AGENDA

This is for you If you are

01	Business Owners	Build your business around Orchard Core
02	Sales / Pre-Sales	Solving your customer pain point
03	Enterprise Architect / Solution Architect	Implement cost effective, audit ready infrastructure
04	Developer	Building Orchard Core Websites and Modules
05	Agency	Provide CMS solutions, Client Website, SEO

The next 60 minutes

01	Beyond the CMS for Enterprises	What Orchard Core lets you build
02	Data and APIs	Storing the domain data and building around it
03	Integrations and workflows	Talking to systems you do not control
04	What we built / The Patterns	A multi-tenant host infrastructure,
05	Takeaways	What to carry back to your codebase

01



Beyond the CMS

Why a modular foundation changes how we build business applications

THE PREMISE

Not just a CMS. An application framework.

Modules, content types and services give enterprise systems the seams they need — the same building blocks that render a marketing page also run your order pipeline.

NEED

What enterprises need

Four common needs for any size of enterprise.

The Platform

Unified Collaboration Platform

Enterprise needs unified collaboration platform

Connected Websites

Connect with Vendors and Customers

Enterprise needs way to connect with customers and Vendors.

SAAS INTEGRATIONS

Heavily invested in SAAS

Often Enterprises use SAAS applications like Microsoft 365, Dynamics, Salesforce, QuickBooks, DocuSign, Dropbox

CLOUD Infrastructure

Heavily Invested in Cloud Platform

Azure, Aws, Google

Why enterprises ❤️ Orchard Core

Four capabilities, all first-class, none of them bolted on afterwards.

DATA

Store the domain

Content types, parts and fields over a JSON document store.
Model the business, not another fixed SQL table.

APIS

Expose and consume

REST/GraphQL over content out of the box, plus your own controllers when the domain needs a custom endpoint.

INTEGRATIONS

Reach other systems

External identity, ERP, CRM, payment — behind typed clients and contracts you own.

WORKFLOWS

Automate the work

Triggers, activities and custom tasks — including pushing documents straight into a document store.

Data: content items are documents

YesSql keeps the whole item as JSON and indexes only what you query.

The document is the truth

A content item is stored whole. No join to reassemble it, no partial writes.

Index what you query, only that

Map and reduce indexes are projections. Add one when a query needs it, not because a field exists.

Parts and fields compose

Adding a field is a content-type change, not a schema migration and a deployment window.

Know where to stop

High-volume transactional data still deserves its own tables. The document store is for the domain, not for everything.

APIs: the application on top

The same content, delivered four different ways.

GraphQL

BUILT IN

Query content types and parts with no endpoint written. Good for read paths and for front ends you do not own.

Your controllers

IN YOUR MODULE

Domain endpoints that speak your language — validate, orchestrate, return your shapes, not the CMS's.

Headless delivery

ANY FRONT END

Orchard Core as the backend; React, mobile or another service consumes it. The admin stays useful for editors.

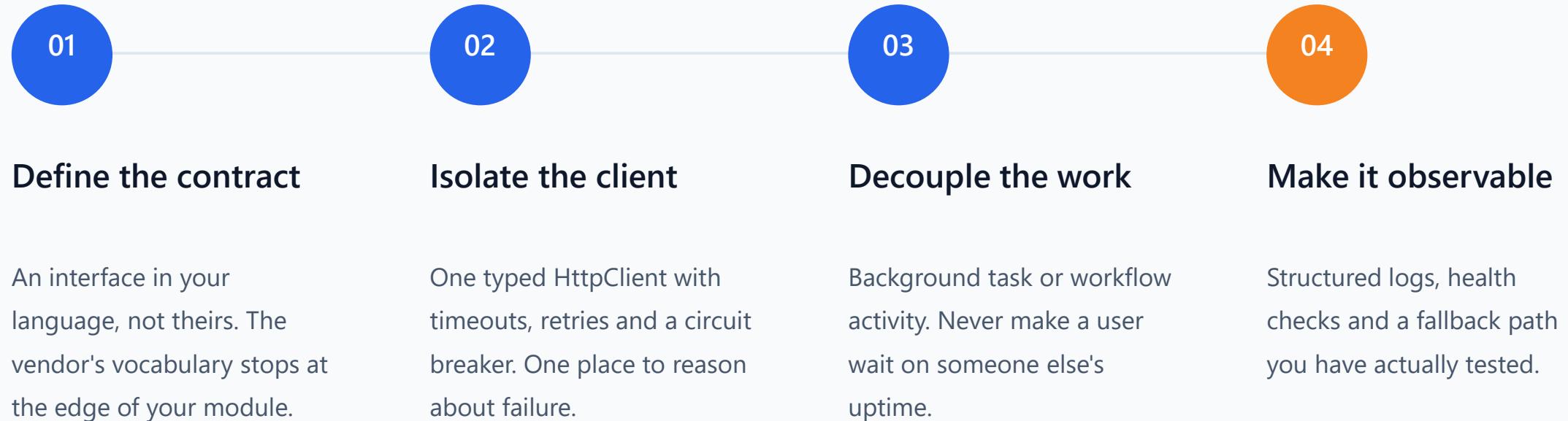
Authentication is the same problem either way — an OpenID server you host, or an external identity provider you federate to.



Integrations and workflows

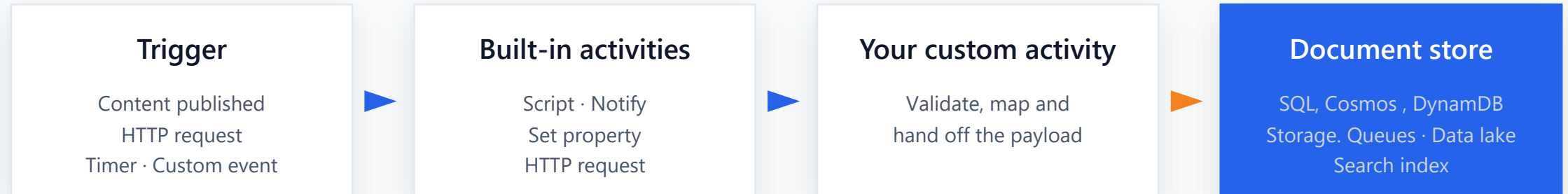
Reaching outside the application, without letting the outside break it

Integrating a system you do not control



Workflows: push data where it needs to go

A workflow is a durable state machine. It survives restarts, so it can wait on things that take time.



Correlation keeps it addressable

Correlate a workflow to a content item or an order id and later events resume the same instance.

The document is already JSON

A content item needs no mapping layer to land in a document database — hand the payload over as it is.

Failure is an outcome, not an exception

Give the activity a Failed branch and the workflow can retry, notify or compensate on its own.

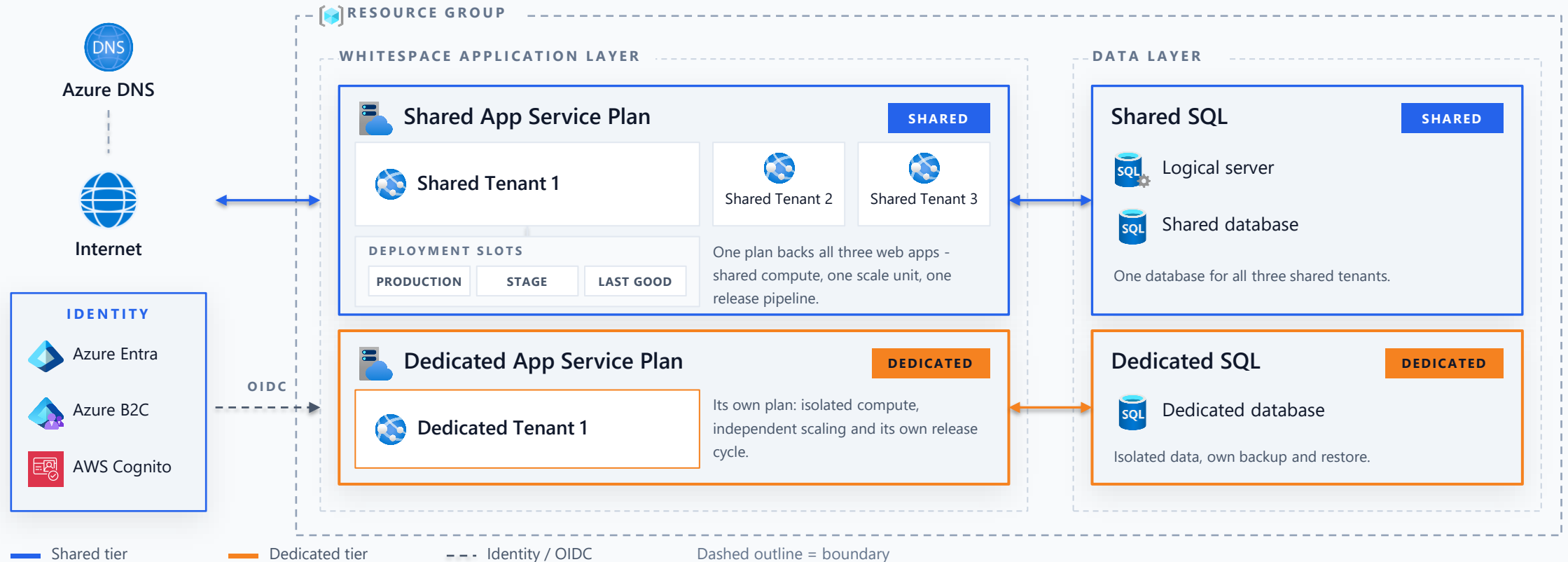


What we built

A multi-tenant Orchard Core host, and the decisions behind it

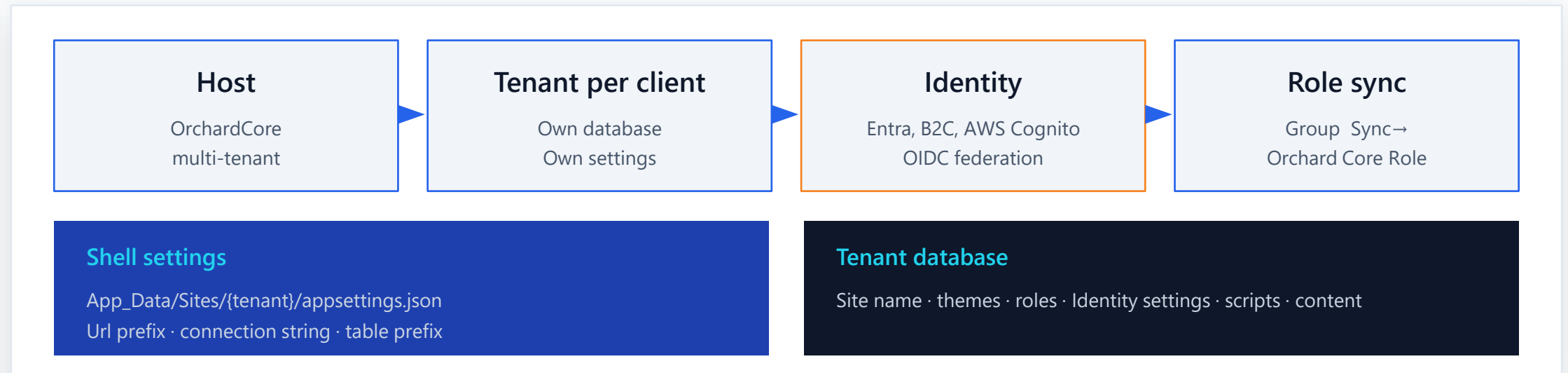
The Infrastructure We Built

WhiteSpace on Azure: three tenants share one plan and one database, one tenant runs on its own.



The host we built

One Orchard Core host, one tenant per client site, external identity for every user.



Orchard Core Host is on Continuous Delivery— CI builds modules NuGet packages, Triggers Host Deployment.

Tenant settings are chore updates – triggers onboarding, reloading of tenants.

DevOps and change management (XOXO) with full audit trail and trackability

From a checklist to a config block

The same outcome, three times, each one less manual than the last.

WAS

A sequence of admin steps

Create the tenant, set up the site, add roles, configure identity, seed the scripts. Per client, by hand, in order.

THEN

One JSON file

A Tenants recipe step that creates the tenant and configures it in one import. Export captures a working tenant.

NOW

A template and a config block

One reusable recipe plus a block of configuration per client. Ten sites are ten config blocks, not ten edited files.

Two operator surfaces — the recipe step and the provisioning

A State Machine & Workflow Automation

Two outcomes, one dependency, and the payload handed straight over.

```
Modules/YourCompany.Workflows/Activities/PushToDocumentStore.cs
```

```
public sealed class PushToDocumentStore : TaskActivity<PushToDocumentStore>
{
    private readonly IDocumentStore _store;

    public override LocalizedString DisplayText => S["Push to document store"];

    public override IEnumerable<Outcome> GetPossibleOutcomes(
        WorkflowExecutionContext ctx, ActivityContext a) => Outcomes("Done", "Failed");

    public override async Task<ActivityExecutionResult> ExecuteAsync(
        WorkflowExecutionContext ctx, ActivityContext a)
    {
        var item = (ContentItem)ctx.Input["ContentItem"];
        await _store.UpsertAsync(item.ContentItemId, item.Content);
        return Outcomes("Done");
    }
}
```

A Composition Via Handlers

Instead of writing monolithic service layer for Domain Events, using PartHandler chain

```
namespace Enterprise.Auditing.Handlers;

public class AuditTrailHandler : ContentPartHandler<AuditTrailPart>
{
    private readonly IHttpContextAccessor _httpContextAccessor;

    // Automatically executes during the Published lifecycle event of ANY content item containing this Part
    public override Task PublishedAsync(PublishContentContext context, AuditTrailPart part)
    {
        var user = _httpContextAccessor.HttpContext?.User?.Identity?.Name ?? "System";

        part.LastModifiedBy = user;
        part.LastModifiedUtc = DateTime.UtcNow;
        part.Apply(); // Merges changes back to the underlying JSON document

        return Task.CompletedTask;
    }
}
```

A Background Task Processing

Run Tenant Scoped Background Task for background processing.

```
namespace Enterprise.Jobs

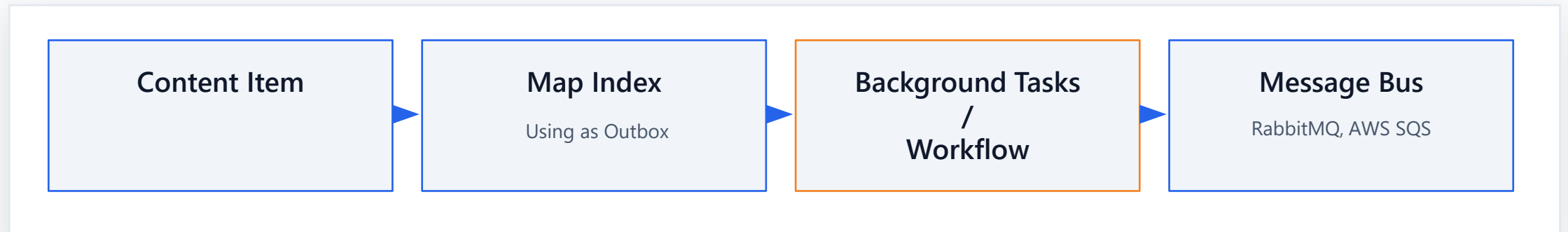
[BackgroundTask(
    Schedule = "*/5 * * * *", // Runs every 5 minutes via Cron
    Title = "ERP Inventory Sync Job",
    Description = "Synchronizes localized inventory per tenant with central ERP.",
    LockTimeout = 10000,
    LockExpiration = 30000
)]
public class InventorySyncBackgroundTask : IBackgroundTask
{
    public async Task DoWorkAsync(IServiceProvider serviceProvider, CancellationToken cancellationToken)
    {
        // Resolves scoped services bound specifically to the current tenant's DI container
        var logger = serviceProvider.GetRequiredService<ILogger<InventorySyncBackgroundTask>>();

        logger.LogInformation("Processing scheduled inventory sync for active tenant...");

        // Add processing logic here
        await Task.CompletedTask;
    }
}
```

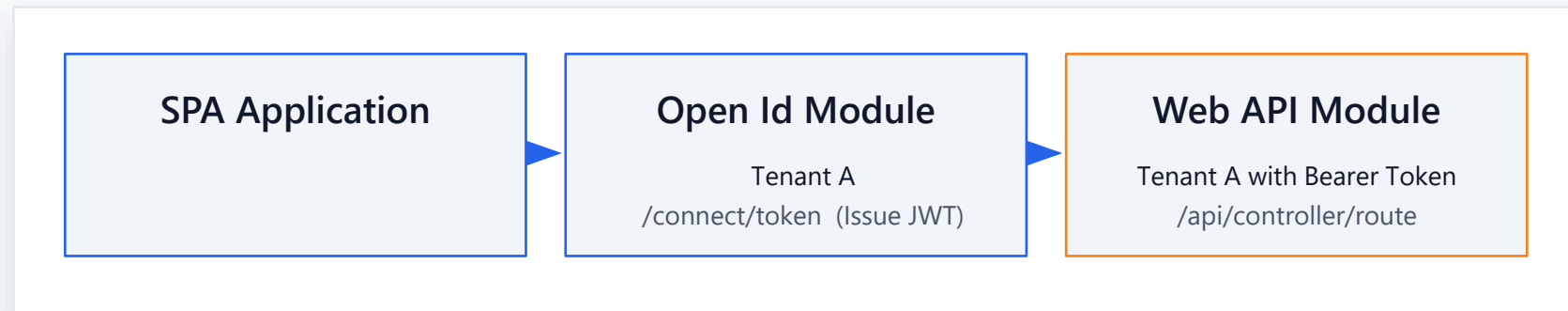
Index as Outbox for Message Bus

Use Content Item, Materialized indexes and Background Tasks, Workflow, combined build, Message Bus / Queue integration



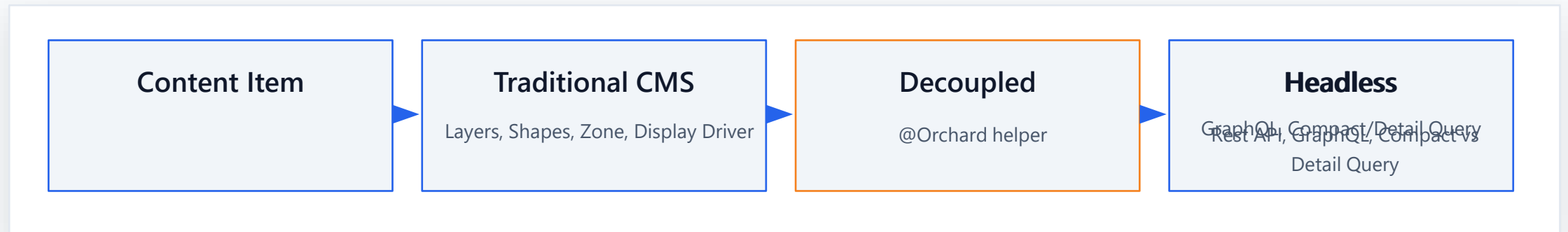
Orchard Core as Authorization Server

Running Orchard Core as decoupled SPA Framework, - React, Angular, Blazor Wasm



UI Patterns, Layers, Shapes, GraphQL

Whether content is consumed in CMS, headless or decoupled, Orchard Covers all



Five things that saved us time and money

01 Removed manual provisioning

No more super user maintenance task to create and manage tenants.

02 Shell settings are auditable

Bundled into source code in mono repo – managed by individual teams.

03 All tenant is Recipe template

Building setup recipes for tenants.

04 Shared Compute / Shared Database / Dedicated App Service

CIO and CFO – are in control of infrastructure expenditure

05 Internal / External Identity

Identity is unique to tenancy tailored to business needs

What to take back to your codebase

01 Model the domain, not the database

Parts and fields keep the schema honest and the intent readable.

02 Draw the seams early

Modules are the boundary you will thank yourself for in year two.

03 Own your integration contracts

The external system will change. Your domain code should not have to.

Start with one module that does one job well. Every other pattern here follows from that seam.

What to take back to your codebase

04 Build UI your way

Weather to build ASP.NET Razor, Blazor or Angular, Build UI your way

02 Authorization unique to tenancy

Roles are in Recipes, Modules implements authorization

03 Cloud Platform Independence

Orchard Core can be built on Azure App Service, Azure AKS, Google GKE or AWS ECS or AWS EKS

Start with one module that does one job well. Every other pattern here follows from that seam.

Questions

Thank you — happy to go deeper on any of these in the chat.

COMPANY surevelox.com

Github <https://github.com/ns8482e>

Slides will be available on my Github account

Niraj Soni

Technical lead and architect
Founder & President
Surevelox
Enriching organizations for success

Orchard Harvest 2026

10–11 September 2026